

CS 3300-A Introduction to Software Engineering

Lecture 01: Introduction & Overview

Dr. Nimisha Roy ▶ nroy9@gatech.edu

Contents

- Course Overview
 - Introduction and Big Picture
- Course Logistics
 - Schedule, Class Organization, Projects, Team Formation, Policies

Your Instruction Team

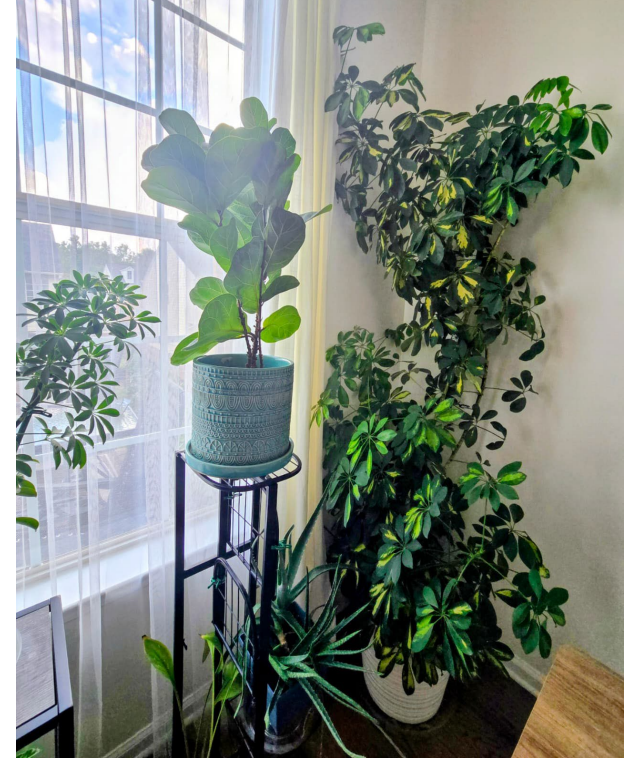


Instructor: Dr. Nimisha Roy

- 5th year teaching this course
- Lecturer in the College of Computing (SCI/OMSA/OMSCS)
- PhD in Computational Science & Engineering from Georgia Tech

Research Interests: Computing Education, Gen AI in education, Accessibility in Education; Sustainability in CS education, Intelligent infrastructure, Software Development

Hobbies: singing (jamming), interior designing, caring for my plants



TA Team



GTA (head TA):
Rishit Patel



UTA:
Jinseo Lee



UTA:
Soham Gunturu



UTA:
Aaron Luu



GTA:
Tianyi Zhang

Course Overview

What is Software Engineering?

What do software engineers actually do?

Software Engineering is the systematic application of engineering principles and practices to design, build, test, deploy, and evolve high-quality software systems.

Software engineers combine programming with engineering practices to solve real-world problems and deliver reliable software.

WHAT DOES THAT INVOLVE?

- | | | |
|----|---|--|
| 01 | Manage the full software lifecycle | Plan → Design → Build → Test → Deploy → Maintain |
| 02 | Work collaboratively | Translate stakeholder needs into software and work effectively in teams. |
| 03 | Apply engineering practices | Use design principles, version control, testing, automation, and modern development tools. |
| 04 | Continuously validate and improve | Test assumptions, evaluate quality, respond to change, and improve the system over time. |

Bottom line: Software engineering is much more than writing code.

Industry Roles

- **Front-End Engineer/Developer:** Creating user interfaces and focus on the visual elements, fixing bugs and user experience (UX).
- **Back-End Engineer/Developer:** Focus on the core logic and performance of the application while taking scalability and integration into account.
- **Full Stack Engineer/Developer:** Handle both front-end and back-end, which means that they create a full application on their own.
- **QA Engineer/Analyst:** Writes the software that tests for quality. QA ensures the product or processes run as expected.
- **DevOps Engineer:** Build and maintain back-end software and distributed systems, such as database servers and application infrastructure.
- **Security Engineer:** Use their ethical hacking skills to test the security of software systems for vulnerabilities that need to be fixed.

What do software engineers do? **The changing landscape**

- **AI-Assisted Development**

GenAI supports design, coding, testing, documentation, and maintenance.

- **Agentic Workflows**

AI agents can plan, execute, test, and iterate on software engineering tasks.

- **AI Governance & Verification**

Engineers must address quality, security, privacy, bias, accountability, and responsible AI use.

- **Evolving Engineering Roles**

More emphasis on problem solving, architecture, verification, and human-AI collaboration - not just writing code.

Takeaway: *AI is changing how software is built - and what software engineers need to know.*

CS 3300 is also embracing the changing landscape.

We will be integrating Gen AI into the Software Engineering process throughout the semester.

How do you benefit from it?

- Practice using AI in real software engineering workflows - not just for generating code.
- Learn when to use AI, when not to, and how to verify its output.
- Develop skills in AI-assisted design, development, testing, and agentic workflows.
- Build engineering judgment around quality, security, responsible use, and AI governance.
- Gain experience with the tools and practices shaping modern software engineering.

What is Software Engineering?

Why is it Important?



What is this ?

- ☐ 4th of July fireworks
- ☐ Flare gun in action
- ☒ Explosion of Ariane 5 rocket due to Software errors

What is Software Engineering?

Why is it Important?

Why is it so difficult to build good software??

Topic of this course and why Software Engineering is an important course in Computer Science

CRASH



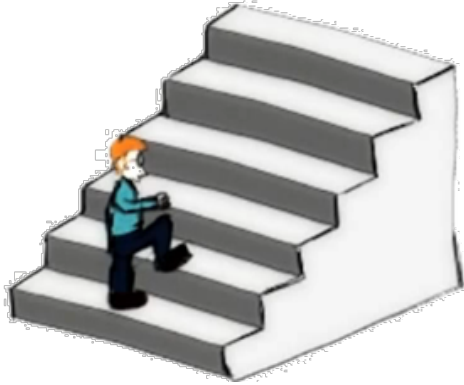
What are attributes of a good software?

- Functionality
 - The software should deliver the required functionality and performance to the user
- Usability
 - Should be easy to use. Not unnecessarily complex
- Maintainability
 - Software must be evolvable to meet changing needs
- Dependability
 - Software must be trustworthy (reliability, security, and safety)
- Efficiency/Productivity
 - Time & Cost effective

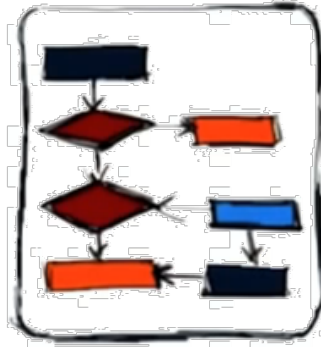
Some questions

- What is the largest software system on which you have worked?
- How many LOCs/day were you producing?
- How many LOCs/day professional software engineers produce?
< 25? 25-50? 50-100? 100-1000? > 1000?
- But what are they doing with the rest of their time?
- How do large, complex software systems get built by many engineers?
- How do teams maintain quality, consistency, and coordination?
 - No one size fits all
 - We will see several

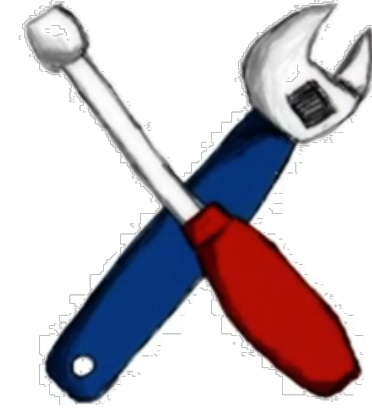
Software Engineering



Methodologies



Techniques



Tools



To build software
of high quality



That works



And fits into
Budget

History

The 60s

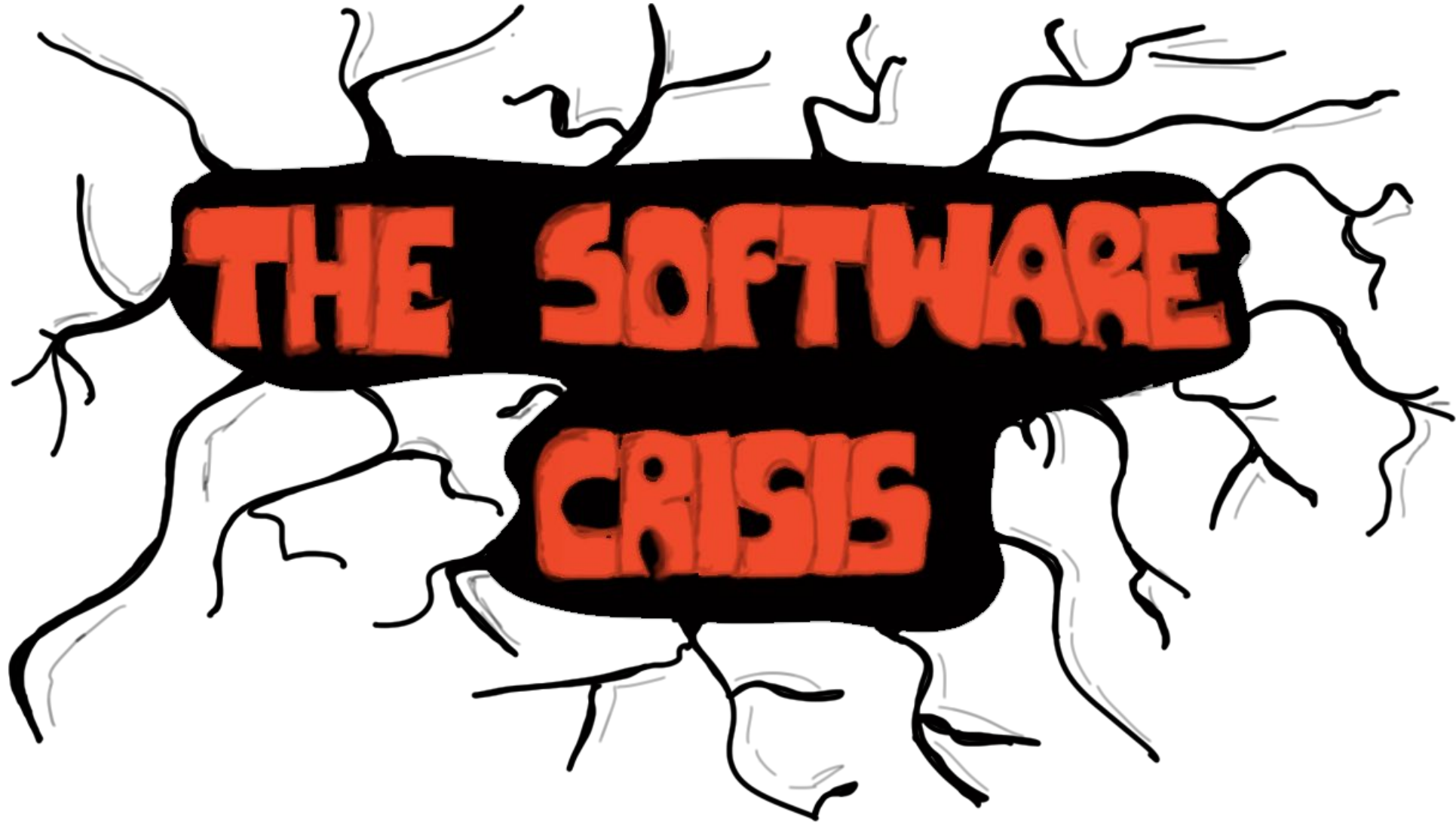


Man on Moon



Polaroid

History

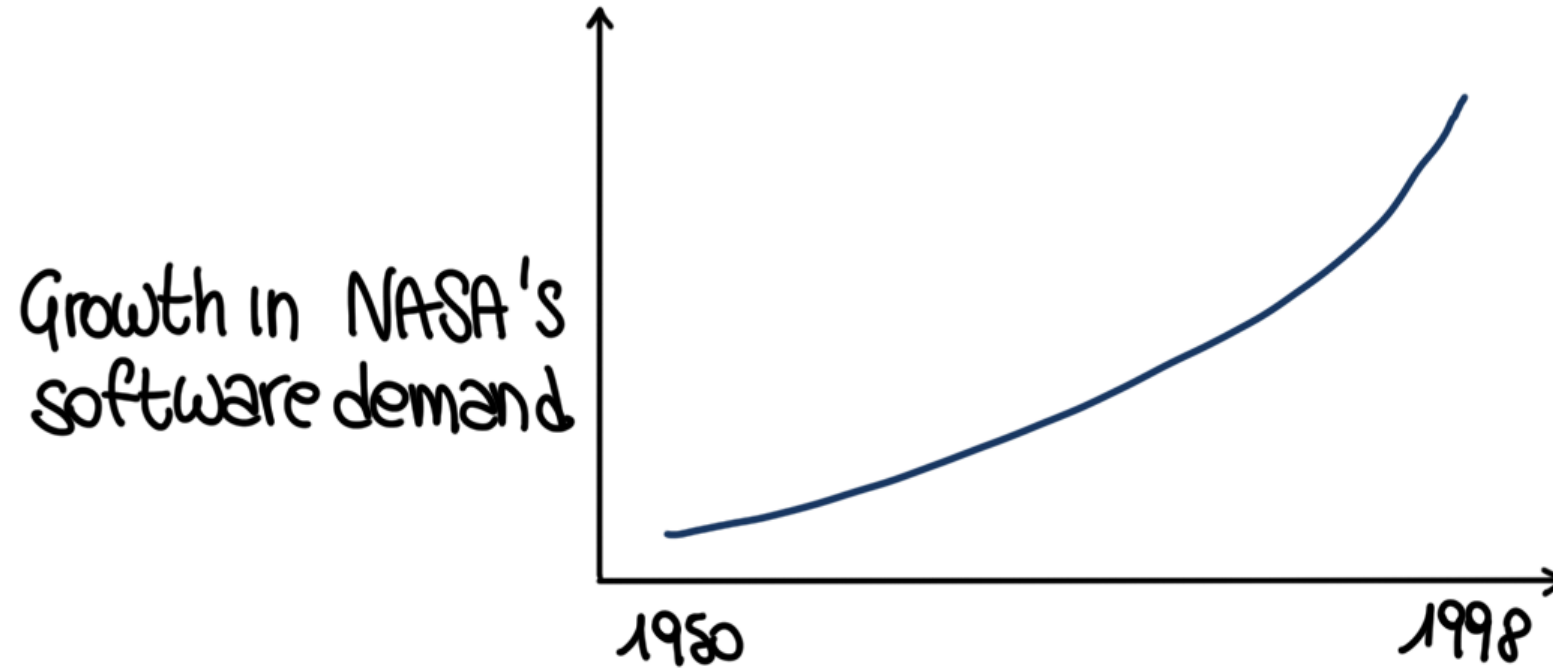


https://en.wikipedia.org/wiki/Software_crisis

History

Reason 1: Rising Demand for Software

HW $\Rightarrow$ SW



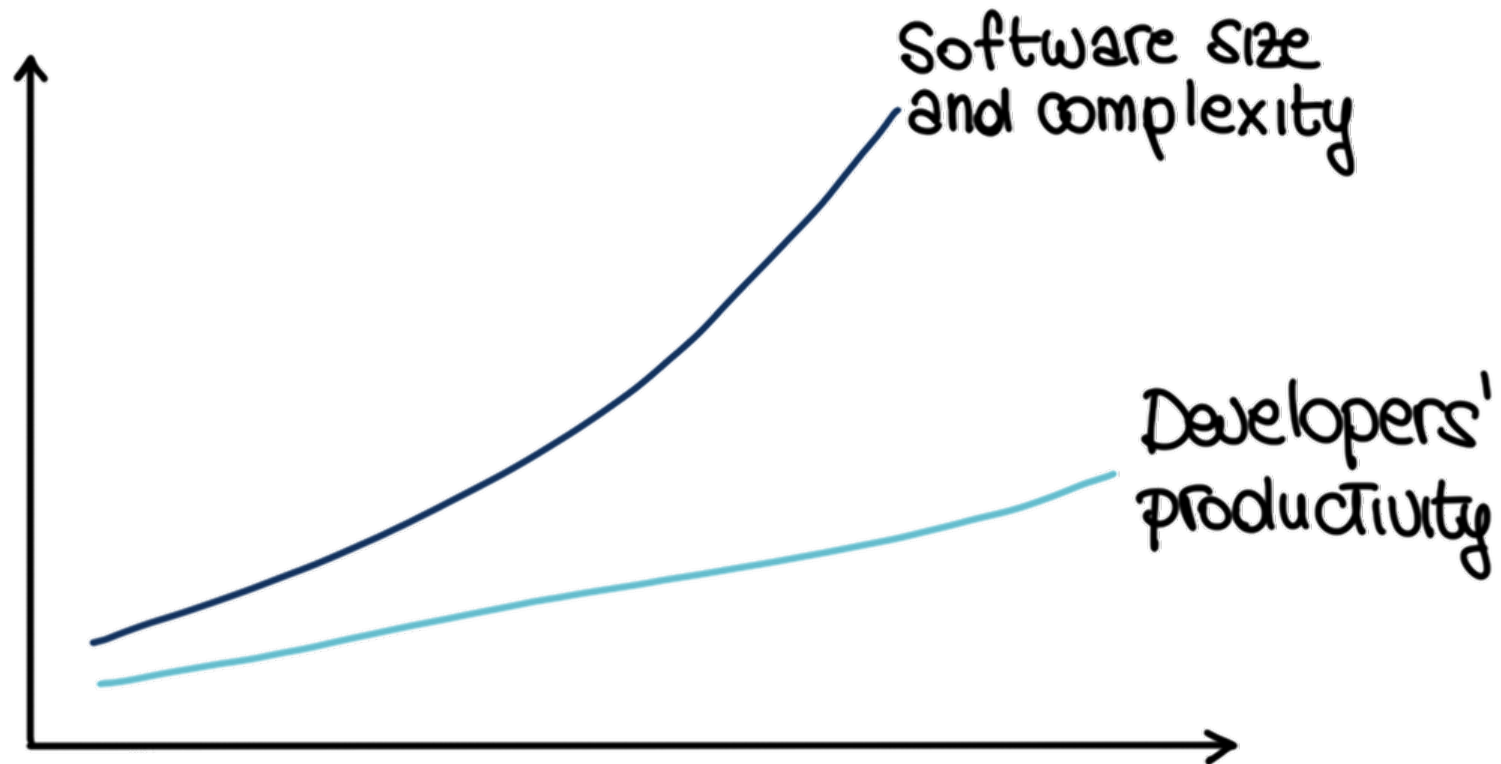
History

Reason 2: Increasing product complexity

SIZE	EXAMPLE	
10^2 LOC	Class exercise	} Programming effort
10^3 LOC	Small project	
10^4 LOC	Term project	
10^5 LOC	Word processor	} Software engineering effort
10^6 LOC	Operating system	
10^7 LOC	Distributed system	
...	...	

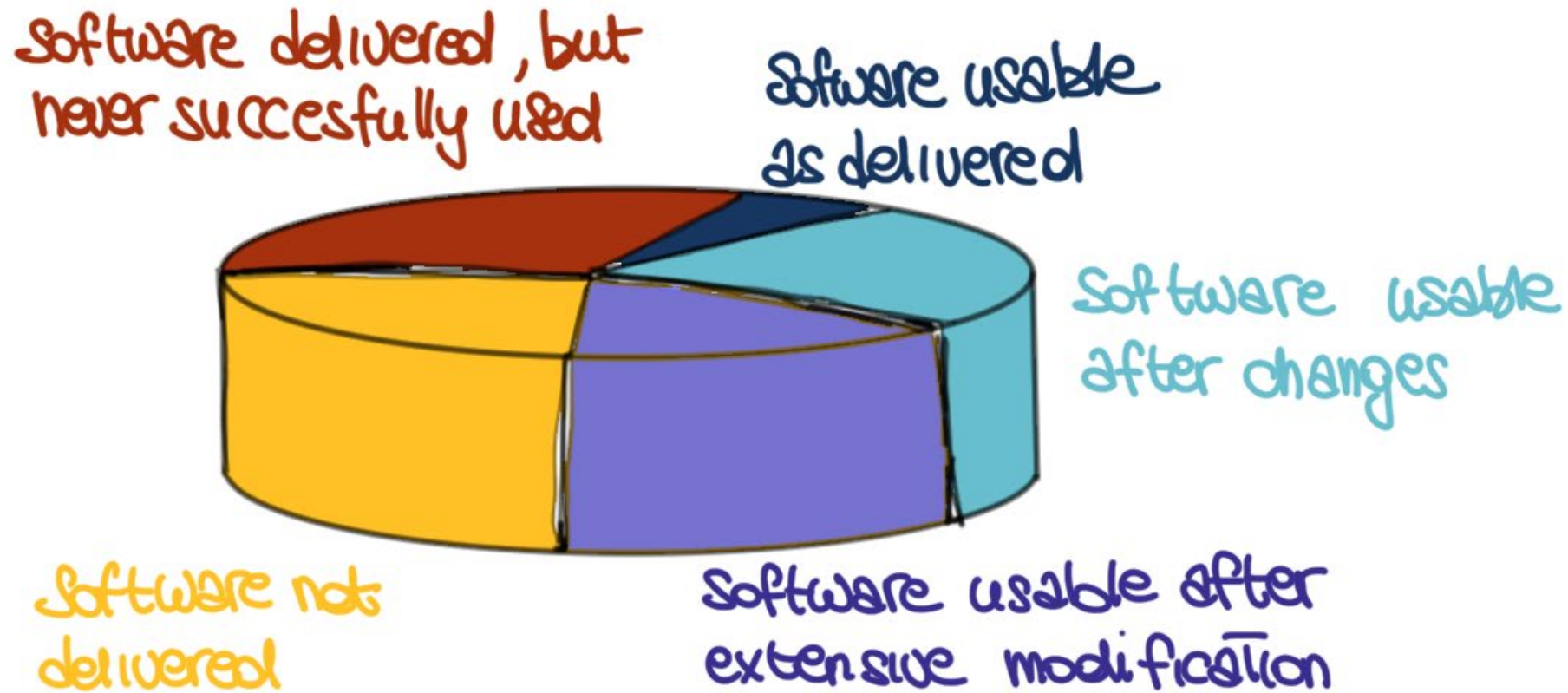
History

Reason 3: Developers Productivity Growth



History

Study of 9 Development Contracts (Davis, 1990)



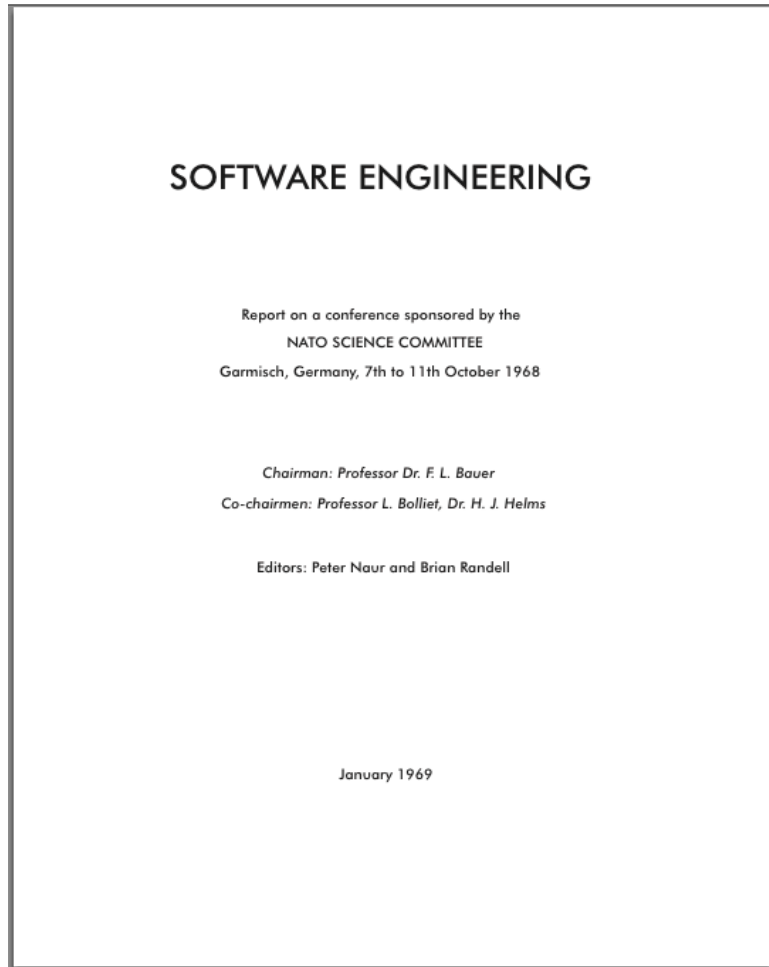
History

Birth of Software Engineering



History

Birth of Software Engineering



Link to proceedings:

<http://homepages.cs.ncl.ac.uk/brian.randell/NATO/>

What are common causes of SW failures?

- No standard procedures for development
- Inadequate understanding of requirements
- Sheer complexity of software (e.g., concurrency, distribution)
- Difficult to match technical knowledge of staff with project needs
- Poor design/implementation/testing methodology
- Requirements change during project
- Poor documentation
- Force fitting software components to applications
- Changing/reusing code without understanding it
- Poor management: lack of communication, poor cost/schedule estimates
- Unrealistic expectations
- Lack of teamwork
- Performance differences among staff

Software importance today

- More and more systems are software-controlled
- The economies of ALL developed nations are dependent on software
- Expenditure on software represents a significant fraction of GNP in all developed countries
- AI technologies integrated into every stage of the software development lifecycle.
- AI-Driven Innovations: Examples include AI-assisted code completion, code generation, test case generation, design diagram generation, predictive analytics in software design, etc.
- Impact on Productivity and Efficiency: AI tools streamline processes, reduce development time, and enhance software quality and accuracy.

What are the key challenges facing SE?

- How can we build high-quality systems?
- How can we do it in a reasonable time?
- How can we do it at a reasonable cost?

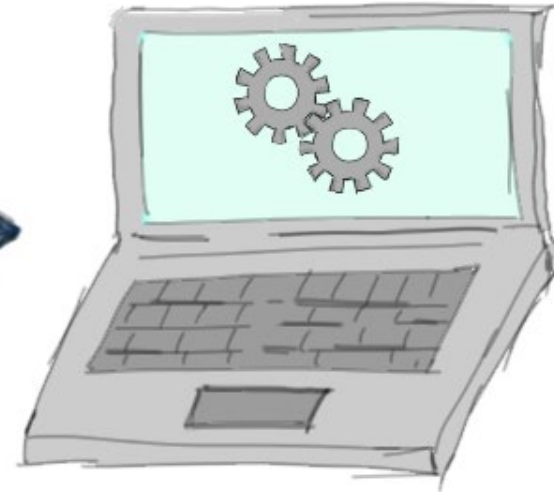
Software Process, Phases, Tools of the Trade

Abstract Idea

System Implementation



Very Complex
Process



Software Process

- Systematic
- Formal

Software Process, Phases, Tools of the Trade

Software Phases



Requirements

Engineering



Design



Implementation



Maintenance



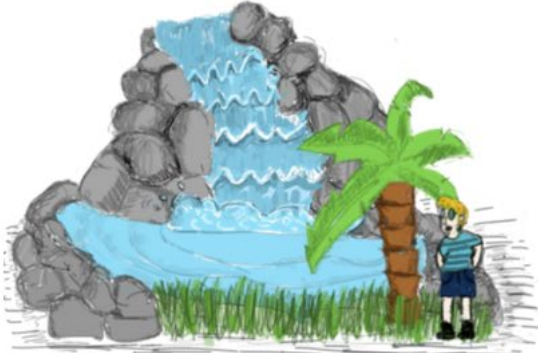
Verification &
Validation

Software Process, Phases, Tools of the Trade

Software Process

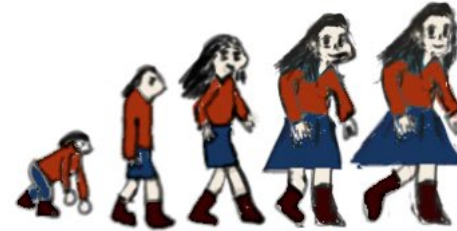
Waterfall

*Sequential,
plan-driven
development*



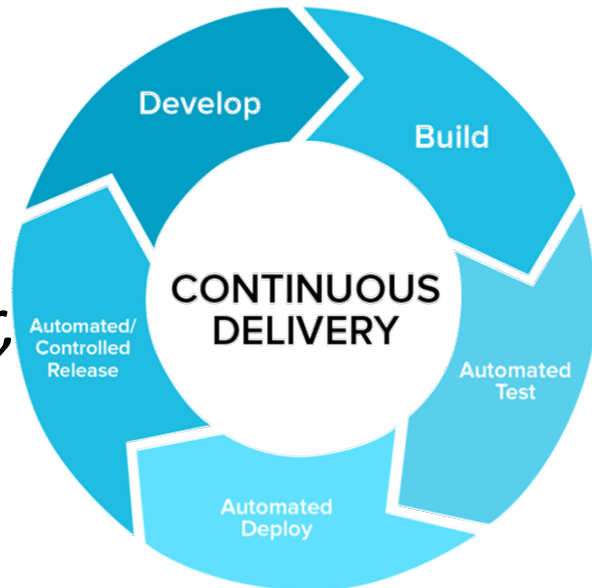
Evolutionary Prototyping

*Build → get
feedback →
refine → repeat*



DevOps/CI

*Integrate, test,
deploy, monitor,
and improve
continuously*



Agile

*Short iterations,
collaboration,
responsiveness
to change*

Course Logistics

Class organization

- Class Website and Canvas
- Ed Discussion
- In-class Lectures
- Attendance is required
- Tools of the Trade Demonstrations – Bring Laptops to class.
- 2 Team projects

Refer to:

Canvas

For anything (syllabus, lectures, helpful links, logistics etc.) related to this class

Ed Discussion

For any communication with the instruction team. No Emails please!

Introduce yourselves on Ed 😊

Grading Distribution

- Assignments (Individual) 20%
- Project 1 (group) 30%
- Project 2 (group) 30%
- Team Management/ CATME Peer Review (Individual) 7%
- Weekly Quizzes (Individual) 10%
- Survey Submissions (Individual) 3%

Tools of the Trade Lectures

Demonstration in Lectures (lets check the living schedule) – Bring your Laptops

- Version Control System (Look at course prerequisite knowledge: You should know basic Git and code review from 2340)
- IDEs
- Front-End Development
- Back-End Development
- Google Cloud Platform
- Agentic SE

Information about projects

- Two projects
- **Project 1:** WEB-APP deployed on Google Cloud Platform. All teams will do the same project. You will use AI tools for implementation and design.
- **Project 2:** You will choose the project. It can be web- or mobile-app. You will use AI tools for implementation, design, testing and an agentic workflow.
- Team based
 - Grades will be adjusted based on peer review
- Tools/environments
 - Project 1 – Springboot, Maven, GCP, GPT 4o API, Copilot, Lucidchart
 - Project 2- GCP but tech stack upto you. AI tools.

AI Tool Integration

- You will choose and experiment with AI tools for implementation, design, and testing.
- You will document your findings according to the metrics provided in the assignments
- Focus is on the basic implementation of software features in projects and more points allocated to your AI integration/reflection aspects
- We are providing primer videos on how to use AI for:
 - Design – lucid chart video on the website (it's free). Many design AI tools are free.
 - Implementation – GPT 4o API credits video on website
 - GPT 4o API credits will be provided to each team
 - Implementation – copilot video on the website (it is free)
 - Testing – video will be published after project 1.
- Each individual will be allocated \$50 (+\$50) worth of Google Cloud credits
- I will be covering basics of prompt engineering next class and agentic TOTT after project 1

Team Formation

- Team Formation Survey:
 - To be completed by *everyone*
 - **Released this Monday; Due: 8/30 at 11:59 PM**
- Team Preferences:
 - Groups of 2+ students: Sign up using Canvas Groups
 - Use the *existing* available group
 - ***Do NOT create your own group***
 - No preferences:
 - Will be matched via CATME

Teams will be formed and mentors by 8/31.

All Project 1 assignments out 8/31.

Summary

Summary

- SE important/critical discipline
 - Concerned with cost-effective software development (all aspects!)
 - Based on a systematic approach that uses appropriate tools and techniques, follows a process, and operates under specific development constraints
- Goal of SE is to deliver high-quality products that provide the expected functionality, meet projected time estimates, and have a reasonable cost



To do this week

- Introduce yourself on Ed
- Submit team formation survey on CATME
- Go through the class website to learn about the schedule
- Go through the 3 AI videos on the class website in the readings column for Day 1
- 1 survey out on the class website for extra credit. It is called AI integration in software engineering. Take 10 minutes to complete the survey now and submit the extra credit assignment.